

Agentic Localization

A technical walkthrough of document-level localization in Sanity, path-based i18n routing in Next.js, and AI-powered translation with glossary and style guide support.

SYSTEM OVERVIEW

How the pieces fit together

Three layers — Sanity Studio, serverless functions, and Next.js frontend — connected by GROQ queries and document functions.

SANITY STUDIO

`l10n.locale`

en-US, de-DE, ja-JP — fallback chains

`l10n.glossary`

Terms + do-not-translate + forbidden

`l10n.styleGuide`

Per-locale brand voice & tone

`article (en-US) ↔ article (de-DE) ↔ article (ja-JP)`

linked via `translation.metadata` — each translation is a separate document

`mark-translations-stale.ts`

Flags translations stale on publish

on: `publish | language == 'en-US'`

`analyze-stale-translations.ts`

AI pre-translates affected fields

on: `update | stale.count > 0 | 120s`

Agent Actions Translate API

`glossary + style guide + protectedPhrases` → AI translation

GROQ queries · `language == $lang`

NEXT.JS FRONTEND

`middleware.ts`

`/ → /[locale]` redirect

`app/[lang]/`

layout + pages + staticParams

`sanityFetch()`

`language == $lang` filter

`LocaleSwitcher` — reads locales from Sanity, swaps `/[lang]/` path segment

CONTENT MODEL

Document-level localization in Sanity

Each translation is a separate document with its own `_id`, linked by `translation.metadata`. Independently publishable, reviewable, and versionable.

Each document gets a `language` field

The `l10n` plugin injects a hidden, read-only BCP-47 language field into every localized type. Editors never see it; queries filter on it.

Locales are content, not config

Each locale is an `l10n.locale` document with code, title, native name, text direction, and fallback chain. Adding a language = creating a document.

Plugin configuration — `sanity.config.ts`

```
const l10n = createL10n({
  localizedSchemaTypes: ['article'],
  // Injects language field, registers l10n types,
  // connects to @sanity/document-internationalization
})
```

Path-based i18n routing in Next.js

Locale as a path prefix — unique URLs per locale, SEO-friendly, CDN-cacheable.

✓ Best practice

/en-US/getting-started

SEO-friendly, CDN-cacheable

✗ Avoid

/getting-started?lang=en-US

Query params ignored by crawlers

1. Middleware

Redirects bare / to /en-US. Sets NEXT_LOCALE cookie.
2. app/[lang]/ segment

All pages under dynamic [lang]. Layout sets <html lang>.
3. generateStaticParams

Exports locale codes for static generation per locale.

```
src/
├── middleware.ts           # / → /[locale] redirect
├── app/[lang]/
│   ├── layout.tsx         # <html lang>, locale switcher
│   ├── page.tsx           # Article list
│   ├── architecture/page.tsx # Architecture page
│   └── [slug]/page.tsx     # Article detail + fallback
```

Querying content by locale

Every GROQ query includes language == \$language. The locale switcher populates from Sanity — no hardcoded list.

```
GROQ queries — queries.ts

*[_type == "article" && language == $language]
  | order(publishedAt desc) { _id, title, "slug": slug.current }

*[_type == "article" && slug.current == $slug && language == $language][0]
  { _id, title, body, "author": author->{ name } }
```

Fallback content strategy

When content isn't translated, the app falls back to the default language with a banner — not a 404.

- Step 1: Query for the article in the requested locale.
- Step 2: If null, re-query with language == "en-US".
- Step 3: Render with "not available in [locale]" banner.
- Step 4: If still null, show 404.

pt-BR → pt-PT → en-US → (404)

Each arrow is a document reference. Editable in Studio — no code changes.

AI-powered translation with context

Not generic MT — context-aware, brand-consistent. Only glossary terms in the source are injected.

Glossary

Approved translations, do-not-translate terms, forbidden terms. Content-filtered per request.

Style Guide

Per-locale brand voice: formality, tone, instructions. DE: formal Sie. JA: desu/masu.

Stale Detection

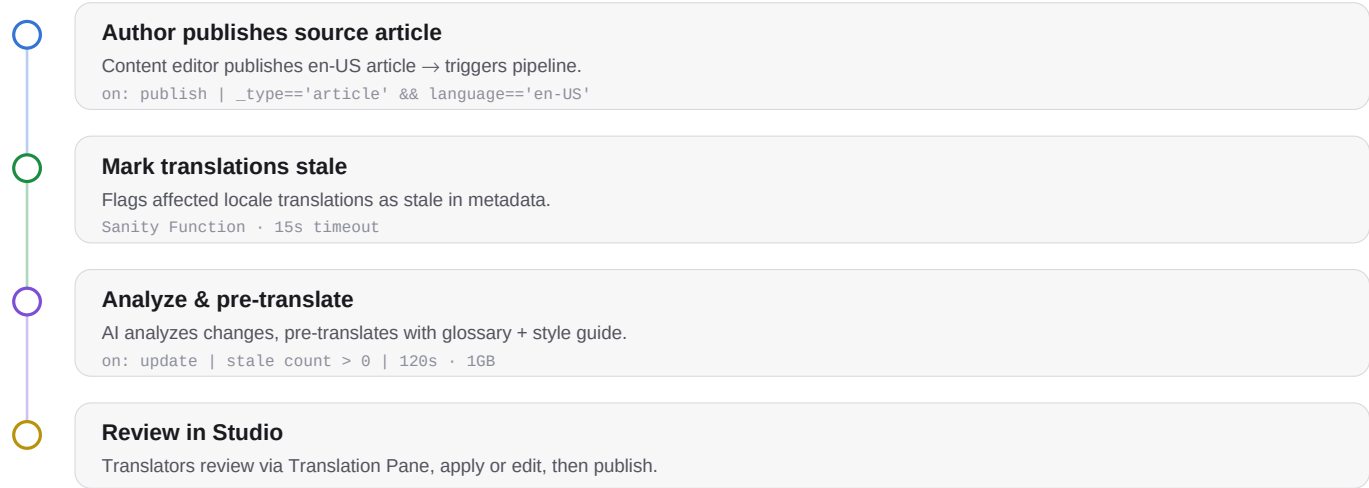
Source changes flag translations stale. AI suggests retranslation vs. cosmetic.

```
Translate API payload — promptAssembly.ts

{ schemaId: "article", documentId: "article-123",
  fromLanguage: "en-US", toLanguage: "de-DE",
  styleGuide: "## Glossary\n- Content Lake → (do not translate)...",
  protectedPhrases: ["Sanity", "GROQ", "Portable Text"],
  targetDocument: { operation: "createOrReplace" } }
```

Automated stale detection pipeline

Defined in sanity.blueprint.ts. Document functions fire on publish/update and run stale detection + AI pre-translation.



Eval: with vs. without context

Score with deterministic checks and an LLM judge. Measure quality delta.

Without context

Source: "Check out our latest Releases"
DE: "...neuesten Veröffentlichungen an"

← Generic translation. Product name lost.

With context

Source: "Check out our latest Releases"
DE: "...neuesten Releases an"

← Glossary: do-not-translate applied ✓

Deterministic checks + LLM judge measure the delta

Project structure

studio/ Studio + translation pane + inspector schemaTypes/ migrations/	:3333	packages/l10n/ Core plugin: schemas, prompts, UI, evals src/schemas/ src/core/ evals/
functions/ Serverless stale detection + pre-translate mark-translations-stale.ts		apps/dashboard/ Translation coverage overview (App SDK)
apps/frontend/ Next.js i18n routing + fallback	:3000	sanity.blueprint.ts Infra-as-code: dataset, CORS, functions
skills/ Claude Code agent skill definitions		